Rajesh Singh

No Collaborators & External Res

#0 : No. of days on previous pset

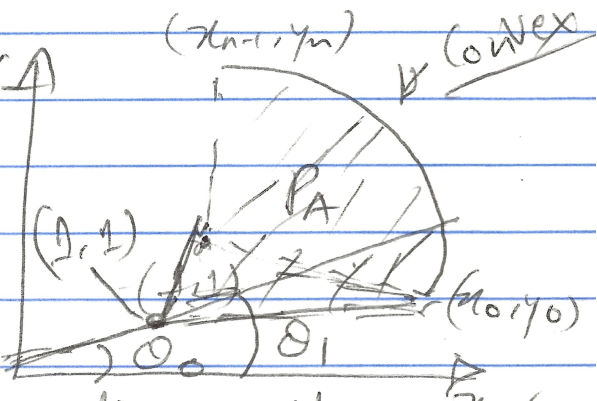
"#" of late days after applying this pset.

p1. CSCI 1200 Rajesh Singh psat 2

1 a) $\mathcal{P} = \{(x_0, y_0) \dots x_{n-1}, y_{n-1}\} \in R^2$ plane
in arbitrary order, interior with $(1,1)$
θ = Area of Polygon = P_A

Since $(x, y) \in R^2$
can be converted in (r, θ)

where $r_0 = \sqrt{(x_0-1)^2 + (y_0-1)^2}$
and θ_0 = angle between



x axis and r line cutting through $(1,1)$ and (x_0, y_0)

Using polar angle to order vertices, and heron formula, the area can be calculated by sorting θ as key and (x, y) coordinate as value)

So, sorting $\{\theta_i, (x_i, y_i)\}$

using merge sort on θ_i

Algorithm would to ~~from~~ form array

~~Area of~~ a) Form array (x coordinate key
(b) Sort on key θ_i
(c) Calculate area

Using Sorting as reduction step

AreaOfConvexPolygon $\leq O(n)$ + Sorting

$\Rightarrow O(n)$ = work to call Sorting algo for "n" vertices

1(b) Algorithm for Calculating Area of Convex

AreaOfConvex ([xi, yi])

n · O(1) → 1. Calculate θ and create array
⇒ O(n) of key (θ) and value (xi, yi)

2. Sort in angle (θi) → $T_{Sorting}(n)$

3. After sorting compute area n · O(n)

So $T_{Area}(n) \leq O(n) + T_{Sorting}(n) + O(n)$

If we chose best comparisim Sorting algorith "MergSort" so

$T_{Sorting}(n) = O(n \log n)$

So, $T_{Area}(n) = O(n \log n)$

CSCI 1200 . Rayesh Singh psdt-2

2. Composition of Reduction

$$\Pi \leq_{T_1(n),\, q_1(n) \otimes h_1(n)} T$$

$$T \leq_{T_2(n),\, q_2(n) \otimes h_2(n)} \Lambda$$

for non decreasing function T_1, T_2, q_1, q_2 and h_1, h_2

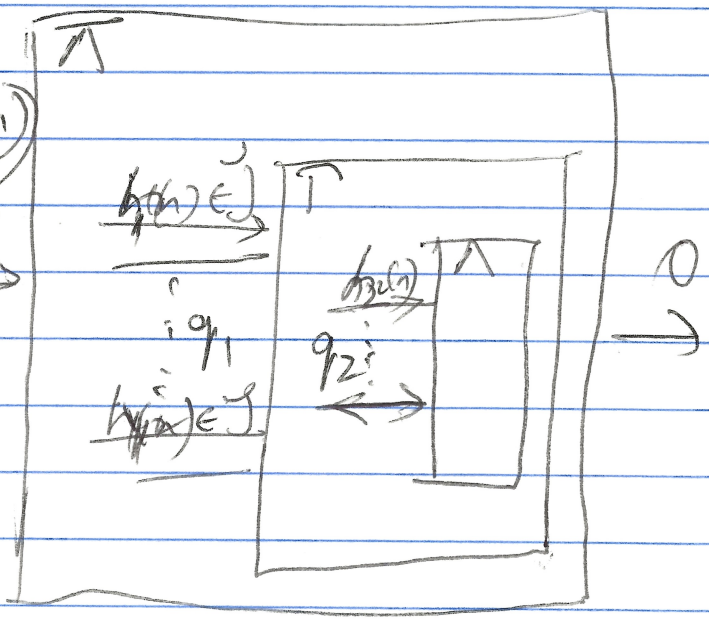
$\Rightarrow$ determine T_3, q_3, h_3 in terms of $T_1, T_2, q_1, q_2, h_1, h_2$ such that $\Pi \leq_{O(T_3(n)),\, q_3 \otimes h_3(n)} \Lambda$

So, $T_3(n) = T_1(n) + q_1(n) \cdot T_2(h_1(n))$

~~$q_3(n) \geq q_1(n)$~~

$q_3 = q_1(n) \cdot q_2(h_1(n))$

$h_3(n) = h_2(h_1(n))$



Consider an input to Π of size(n)
First reduction performs $T_1(n)$ work with $q_1(n)$ calls to T, each on input size $m \leq h_1(n)$, so
So, second reduction performs:

2. Contd....

2. Second reduction performs,
- at most $T_2(m) \leq T_2'(h_1(n))$
and makes at most $q_2(m) \leq q_2(h_1(n))$ calls to $\wedge$
each input to $\wedge$ of size $\leq h_2(m) \leq h_2(h_1(n))$

This inequality is due to non-decreasing func.
Multiply the work and call count per T call by the number $q_1(n)$ of such calls give T_3 and q_3

Correctness is proven because second reduction provide correct answer to T query, so first reduction still produce valid answer to Π.

Thus, $\Pi \leq O(T_3(n), q_3(n) \otimes h_3(n))$ $\wedge$

p-5 CSCI-1200, Rajeet Singh-, pset 2.

3(a) "Select" method has bug. It incorrectly calculated the index when recursing into the right subtree. I fixed it by subtracting the size of (left-subtree+1) for current node only when recursing right, since left-subtree indices don't need adjusting. "Insert" was corrected - explanation on next page.

3(b) Let bstRotate(v) be the usual rotation that moves a non-root node "v" above its parent "p". After rotating update the size of the node moved down before updating the node moved up.

```
size(u):
    if u == none:
        return 0
    return u.size
rotateWithSize(v):
    p = v.parent
    bstRotate(v)
```

p.size = 1 + size(p.left) + size(p.right)
v.size = 1 + size(v.left) + size(v.right)

This works for both left and Right rotation. The ordinary rotation takes O(1) and extension add only 2 constant

(p-6) CSCE-1200 Rajesh Singh pset 2

3(a) "Insert" method didn't update the size of the ~~node~~ tree after insertion.
The corrected version replaces the call to calc size with self.size+=1 otherwise it will end up calling self.calculate_size() costing O(n) instead of O(h)